

# Experiment 2 — Client-Server Communication Between Two Virtual Machines

## Cloud Computing Lab

|                    |                                                     |
|--------------------|-----------------------------------------------------|
| <b>Name</b>        | Deep Shekhar Halder                                 |
| <b>Roll Number</b> | UG/02/BTCSE/2023/063                                |
| <b>Course</b>      | B.Tech CSE, Adamas University                       |
| <b>Subject</b>     | Cloud Computing Theory and Lab                      |
| <b>Faculty</b>     | Anusuya Bera, Assistant Professor (CSE, SOET)       |
| <b>Experiment</b>  | Lab 2 — Client-Server Communication Between Two VMs |

## 1. Objective

Configure two virtual machine instances in VirtualBox to establish client-server communication and implement a basic TCP-based chat application between the two virtual machines.

## 2. Tools Used

- Hypervisor: Oracle VirtualBox
- Guest OS: Ubuntu (both VMs)
- Programming Language: Python 3 (socket programming)
- Host OS: Windows

## 3. Network Configuration

- VM 1 (Server): Ubuntu-CloudLab — IP: 10.0.2.15
- VM 2 (Client): Ubuntu-Client — IP: 10.0.2.3
- Network Type: Custom NAT Network named 'LabNet'
- Subnet: 10.0.2.0/24
- Both VMs attached to the same NAT Network with unique MAC addresses

## 4. Steps Performed

1. Cloned the existing Ubuntu VM to create a second VM (Ubuntu-Client).
2. Created a custom NAT Network named 'LabNet' (subnet 10.0.2.0/24) via VirtualBox's Network Manager.
3. Attached both VMs' network adapters to the 'LabNet' NAT Network.
4. Regenerated the MAC address on the cloned VM to resolve an IP address conflict.
5. Verified each VM's IP address using the 'ip a' command.
6. Tested connectivity between the two VMs using the 'ping' command in both directions.
7. Designated Ubuntu-CloudLab as the Server and Ubuntu-Client as the Client.
8. Wrote a TCP server program (server.py) on the Server VM using Python's socket module.
9. Wrote a TCP client program (client.py) on the Client VM using Python's socket module.
10. Ran the server first, then the client, establishing a TCP connection between the VMs.
11. Exchanged messages interactively between the Server and Client to demonstrate the chat application.
12. Closed the connection cleanly by sending 'bye' from the client side.

## 5. Screenshots

### 5.1 Custom NAT Network Configuration

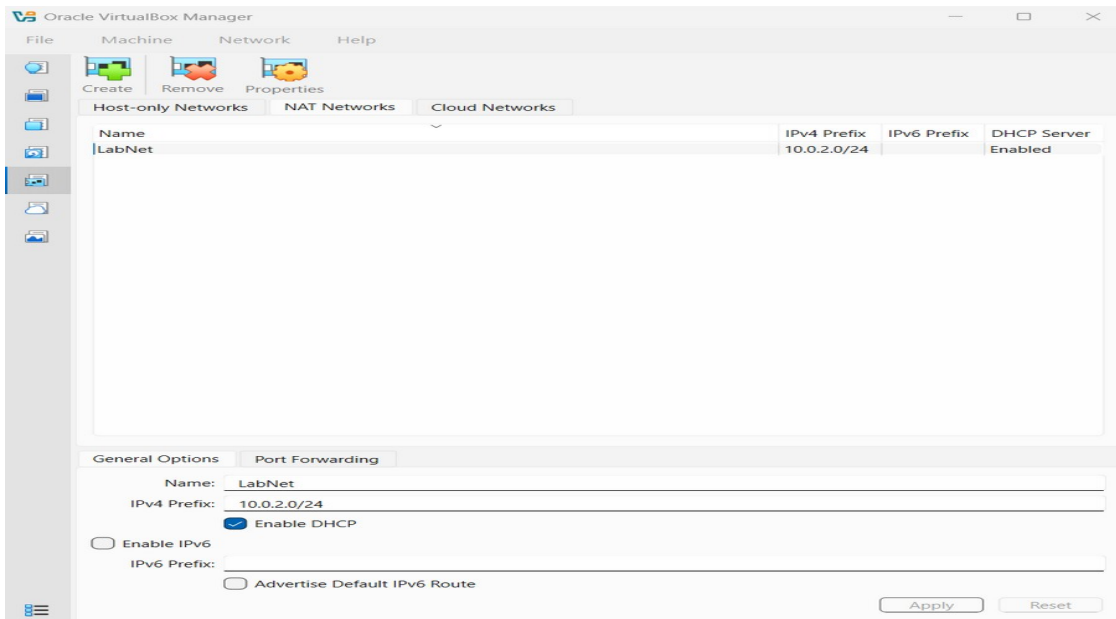


Figure 1: Custom NAT Network 'LabNet' created with subnet 10.0.2.0/24 and DHCP enabled.

### 5.2 IP Address Configuration on Both VMs

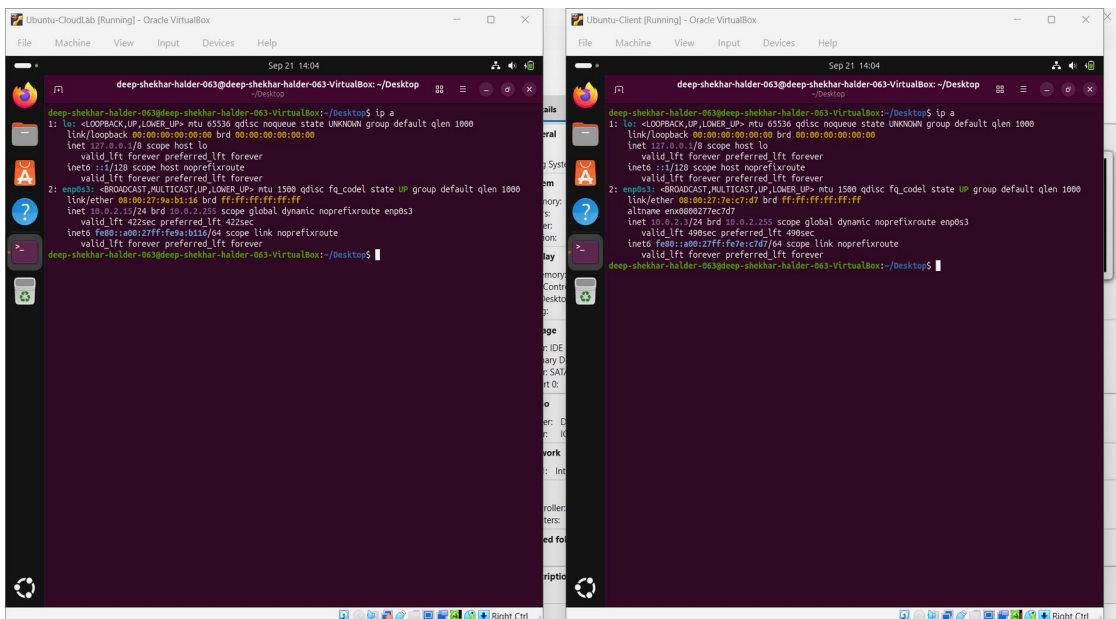


Figure 2: 'ip a' output showing Ubuntu-CloudLab at 10.0.2.15 and Ubuntu-Client at 10.0.2.3, each with a unique MAC address.

### 5.3 Connectivity Test (Ping)

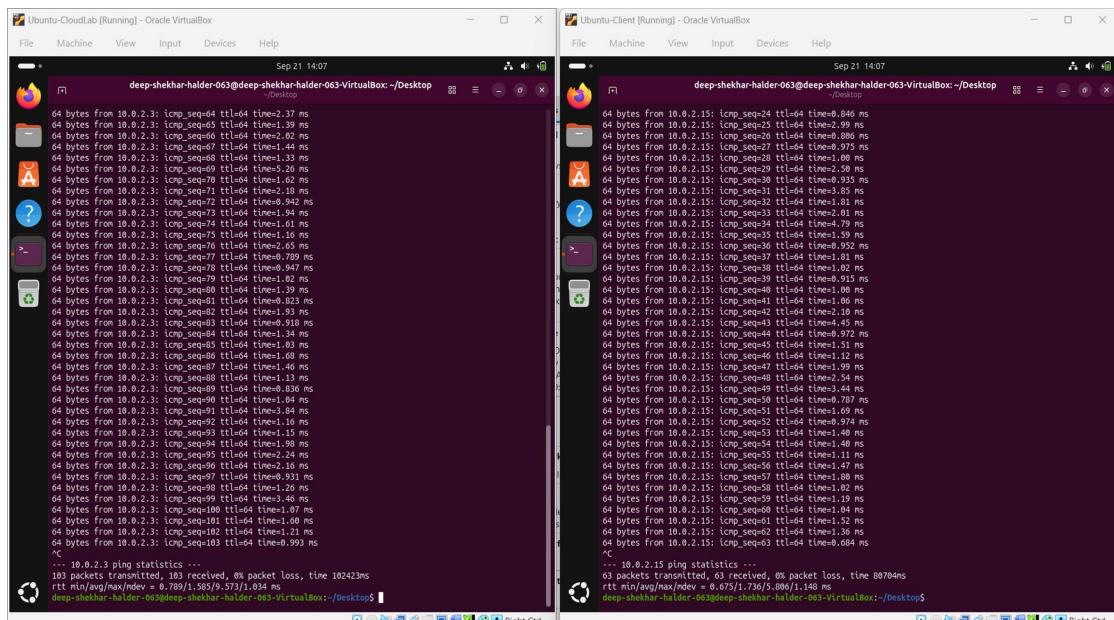


Figure 3: Successful ping between both VMs with 0% packet loss in both directions.

## 5.4 TCP Server and Client Source Code

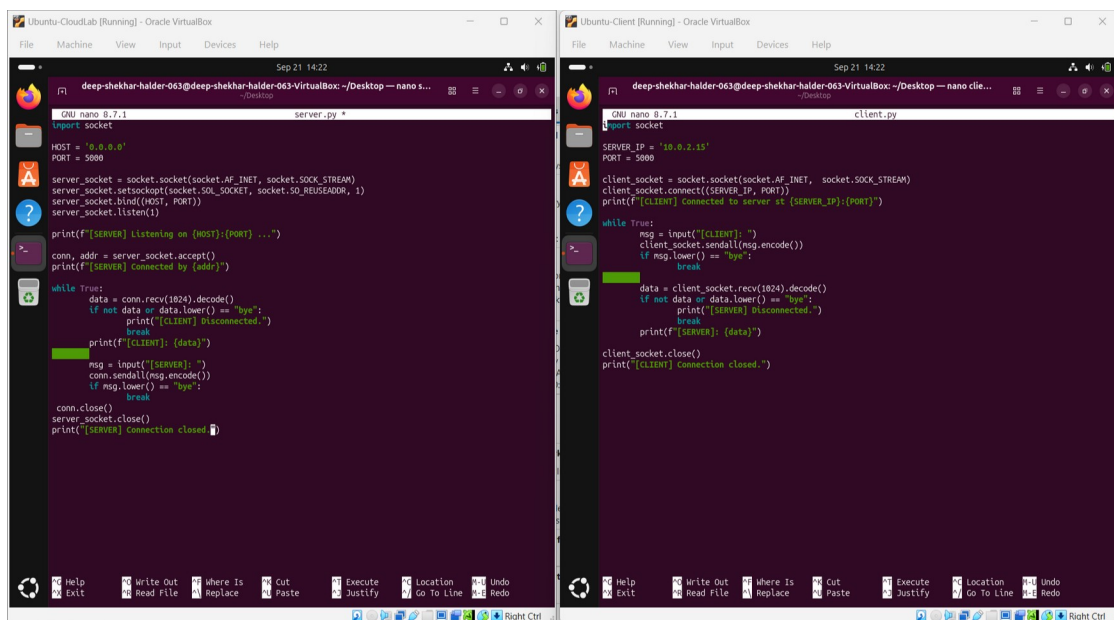


Figure 4: `server.py` (left) running on Ubuntu-CloudLab and `client.py` (right) running on Ubuntu-Client.

## 5.5 Server-Client Connection Established

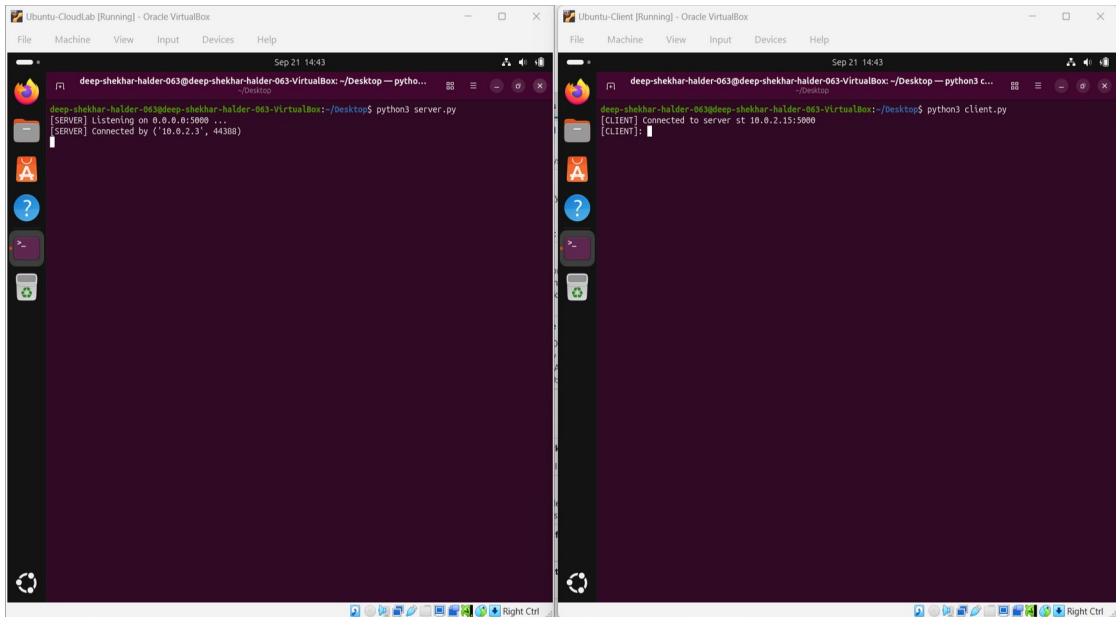


Figure 5: Server listening and accepting a connection from the Client; Client successfully connected to the Server.

## 5.6 Chat Communication and Clean Disconnection

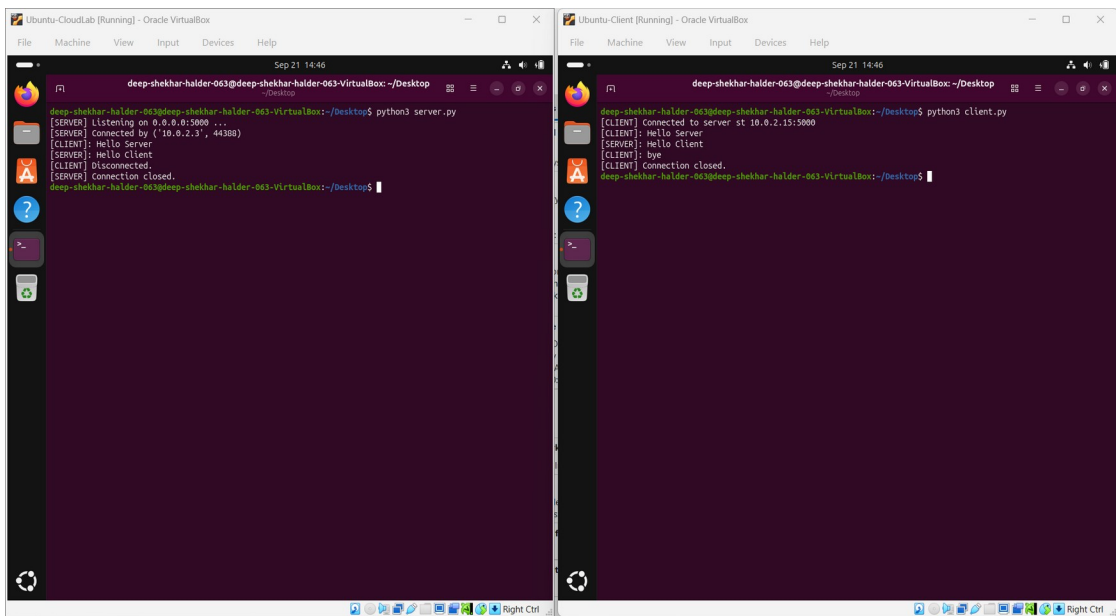


Figure 6: Full chat exchange between Server and Client, ending with 'bye' and a clean connection closure on both sides.

## 6. Conclusion

Two Ubuntu virtual machines were successfully configured on a common NAT Network within Oracle VirtualBox. Connectivity between the two VMs was verified using the ping command, confirming reliable communication with zero packet loss. A TCP-based client-server chat application was implemented in Python, with one VM acting as the

Server and the other as the Client. Messages were successfully exchanged in both directions, and the connection was terminated cleanly, demonstrating a fully functional client-server communication setup between two virtual machines.